Facultad de Ingeniería
Departamento ingeniería en sistemas
Manejo de archivos
Sección 01
Docente: Manuel Gustavo Rojas Vasquez

Proyecto 2
Agenda de Contactos con Acceso Aleatorio

Javier André Huertas Bravo - 1505825

Quetzaltenango, 9 de noviembre de 2,026

1. Estructura del registro de tamaño fijo

Para crear él proyecto se diseñó una estructura de registro de longitud fija equivalente a exactamente 100 bytes codificados en estándar UTF-8. Cada campo se rellena por la derecha con espacios en blanco (carácter ASCII `0x20`), la cual valida la longitud a nivel de bytes para evitar que caracteres multibyte como vocales tildadas o la letra «ñ» desfasen el alineamiento de los registros subsecuentes.

Campo	Tipo lógico	Rango de bytes	Tamaño	Regla de Relleno y validación
id	Entero unico	Bytes 0-9	10 bytes	Entero positivo convertido a cadena
nombre	Cadena UTF-8	Bytes 10-39	30 bytes	Recorte seguro a 30 bytes
teléfono	Cadena UTF-8	Bytes 40-54	15 bytes	Almacena números locales o con codigo de area rellenados a 15 bytes
correo	Cadena UTF-8	Bytes 55-79	25 bytes	Dirección electrónica ajustada a 25 bytes mediante relleno de espacios
ciudad	Cadena UTF-8	Bytes 80-99	20 bytes	Localidad geográfica ajustada a 20 bytes
total	Registro fijo	Bytes 0-99	100 bytes	Sin saltos de linea intermedios

2. Implementación de Acceso Aleatorio con seek() y tell()

Él acceso aleatorio evita recorrer secuencialmente él archivo desde él principio. Como todos los registros miden exactamente él tamaño récord que son 100 bytes, la posición física de cualquier contacto se calcula multiplicando su número de registro por 100.

Él método f.seek() mueve él puntero de lectura/escritura del sistema operativo directamente al byte calculado, mientras que f.tell() retorna la ubicación actual en bytes para verificar la operación y mostrarla en la barra de estado de la interfaz

3. Funcionamiento y eficiencia del indice primario

Él archivo indice.json almacena un diccionario clave-valor donde la clave es él id único del contacto y él valor es su posición de registro (0,1,2,...,n). Al iniciar él programa, este índice se carga en memoria RAM como un diccionario hash de python. Cada vez que se agrega un contacto al final de contactos.dat, se divide él byte devuelto por f.tell() entre 100 para obtener él número de registro t se actualiza inmediatamente indice.json.

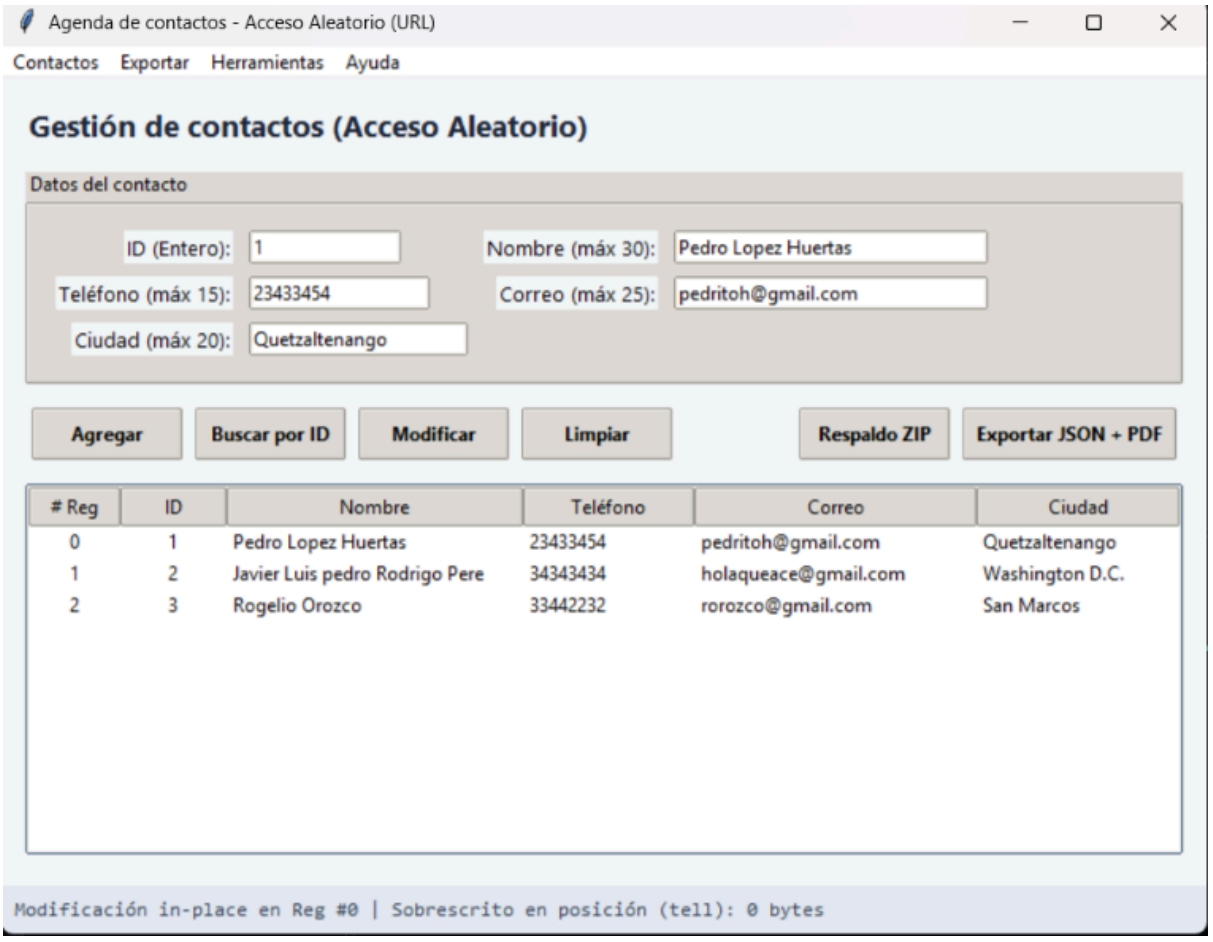
Una búsqueda apoyada en índice es más eficiente que una búsqueda secuencial porque desacopla la localización de la clave respecto a la lectura en disco. En una búsqueda secuencial sin índice, encontrar un contacto entre N registros requiere leer y analizar en promedio N/2 bloques de disco (y hasta N en él peor caso, cuando él ID no existe).

4. Comparación del tamaño original contra él respaldo ZIP

La herramienta de respaldo empaqueta los archivos contactos.dat e indice.json dentro de un contenedor comprimido mediante él algoritmo zip_deflated de la libreria estándar zipfile.

Archivo / Metrica	Tamaño original (Bytes)	Tamaño en respaldo ZIP	Porcentaje de reduccion
Contactos.dat [(N)] registros * 100	342 bytes	420 bytes	22.81%
indice.json	342 bytes	420 bytes	---
Suma total evaluada	342 bytes	420 bytes	----

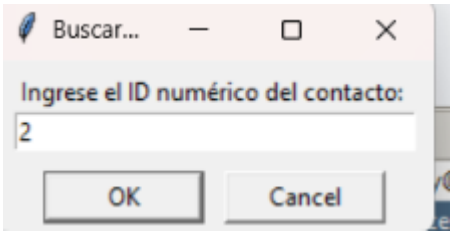
5. Capturas de pantalla programa funcionando



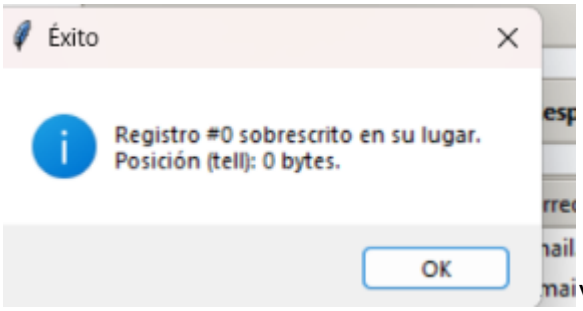
Inicio del programa



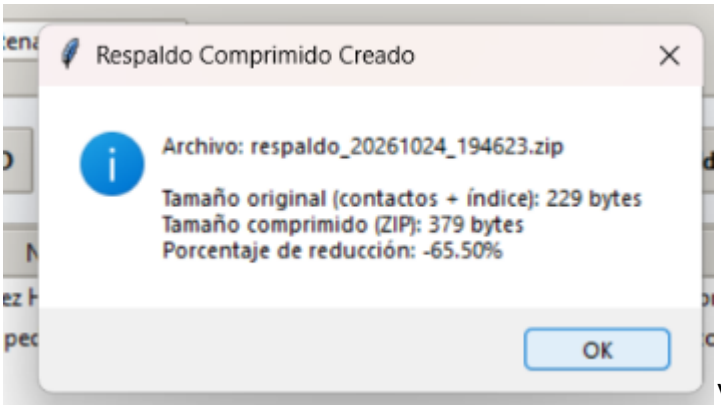
Mensaje al agregar un contacto



Ventana para buscar por ID



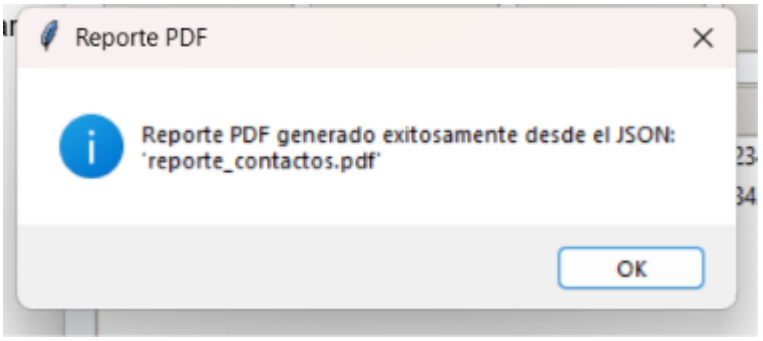
Ventana al modificar un contacto



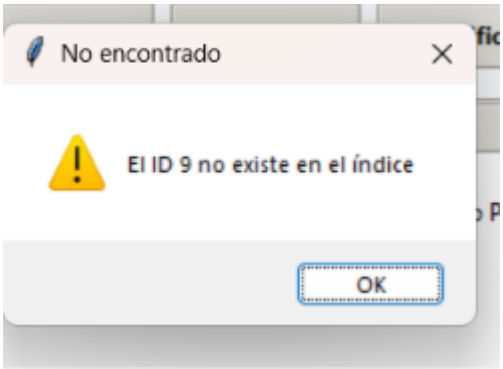
Ventana al crear él archivo ZIP



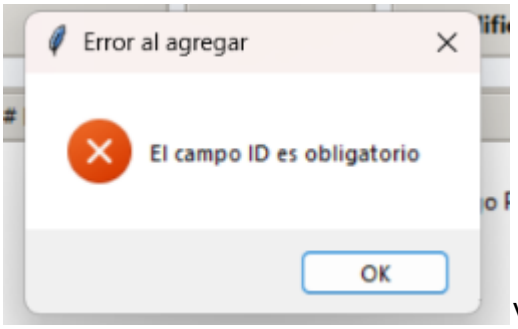
Exportar él json y él PDF



Mensaje de éxito al generar pdf desde json



Ventana si no existe un ID



Ventana de error al agregar un contacto

Reporte de Agenda de Contactos

Fecha de generacion: 2026-11-07 17:58:03

ID	Nombre	Telefono	Correo	Ciudad
1	Pedro Lopez Huertas	23433454	pedritoh@gmail.com	Quetzaltenango
2	Javier Luis pedro Rodrigo Pere	34343434	holaqueace@gmail.com	Washington D.C.
3	Rogelio Orozco	33442232	rorozco@gmail.com	San Marcos
67	Bernardo Arevalo	34434555	TioBernieGt@yahoo.es	Ciudad De Guatemala

Reporte de contactos generado